



SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week1 – Monday	Batch	23CSBTB47B
Name	Manoj Aelishala	Hall Ticket No	2303A54054
Assignment Number:3.1			

QUESTION 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

PROMPT:

Write a Python function that checks if an integer is a palindrome using arithmetic only (no string conversion). Return True if it is, otherwise False.

SCREENSHOT OF GENERATED CODE:

```
def is_palindrome(num: int) -> bool:
    # Negative numbers are not palindromes
    if num < 0:
        return False

    original = num
    reversed_num = 0

    while num > 0:
        digit = num % 10
        reversed_num = reversed_num * 10 + digit
        num //= 10

    return original == reversed_num

print(is_palindrome(121)) # True
print(is_palindrome(-121)) # False
print(is_palindrome(12321)) # True
print(is_palindrome(123)) # False
```

INPUT & OUTPUT:

Input	Output	Reason
121	1	Reversed number is 121, which is equal to the original number.
-121	0	Negative numbers are not considered palindromes.
10	0	Reversed number is 01 (1), which is not equal to 10.
12321	1	Reversed number is 12321, which matches the original number.

EXPLANATION:

The function first returns False if the number is negative. It then saves the original number and reverses it digit by digit using a loop. Finally, it compares the reversed number with the original and returns True if they are equal, otherwise False.

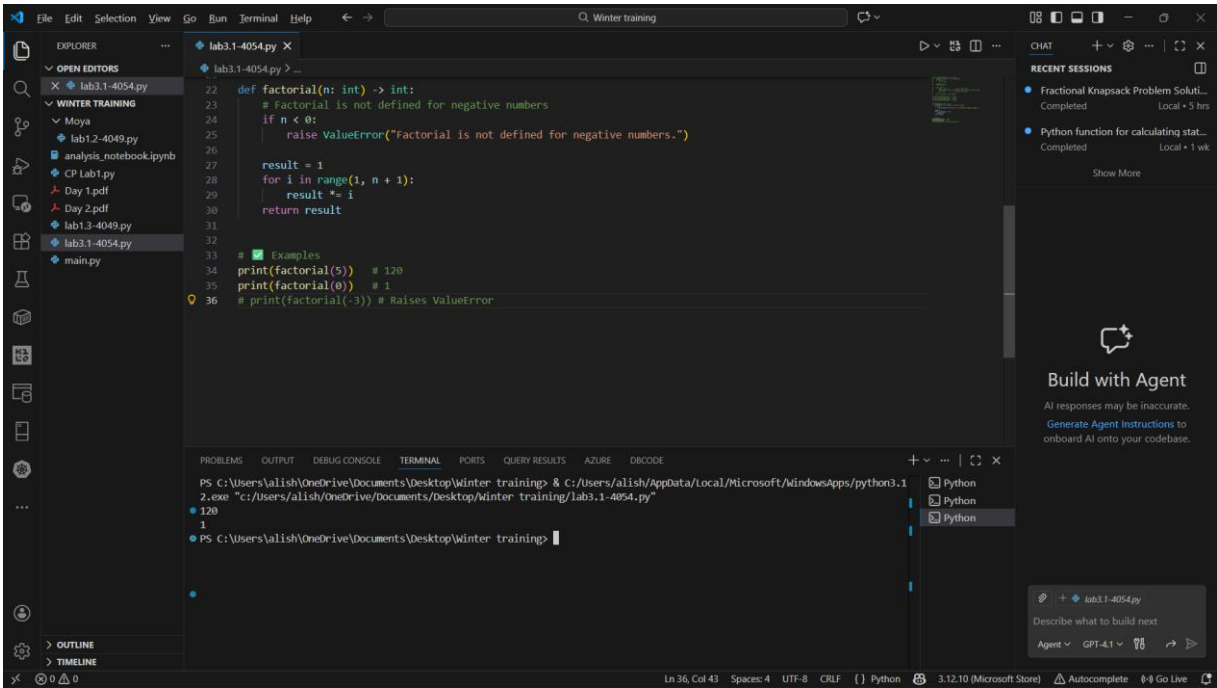
Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

PROMPT:

Write a Python function that computes the factorial of a given integer. The function should return the factorial value for non-negative inputs and handle invalid cases (e.g., negative numbers) appropriately.

SCREENSHOT OF GENERATED CODE:



INPUT & OUTPUT:

Input	Output	Reason
5	120	$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120.$
0	1	By definition, $0!$ is equal to 1.
-3	Invalid input	Factorial is not defined for negative numbers.
7	5040	$7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 5040.$

EXPLANATION:

The factorial of a number is the product of all positive integers up to that number. By definition, $0!=1$, and factorials are not defined for negative numbers

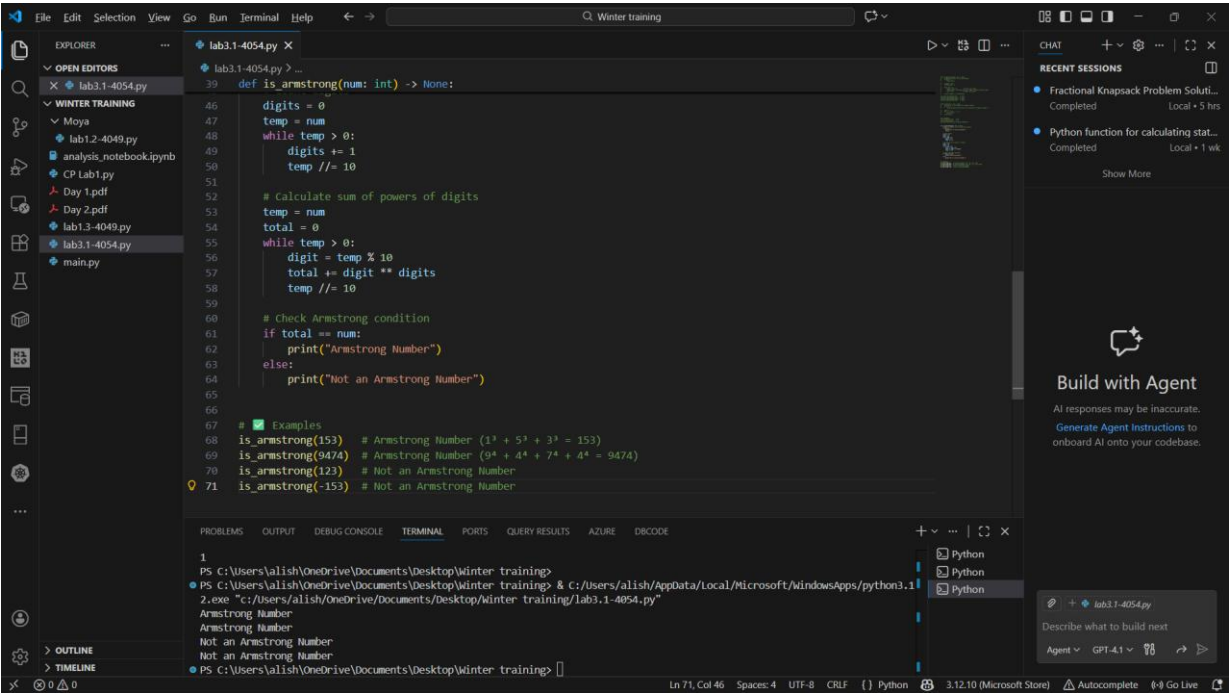
Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

PROMPT:

Write a Python function that checks if a given integer is an Armstrong number. The function should print "Armstrong Number" if the input satisfies the condition, otherwise print "Not an Armstrong Number".

SCREENSHOT OF GENERATED CODE:



INPUT & OUTPUT:

Input	Output	Reason
153	Armstrong Number	$(1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153)$, which equals the original number.
123	Not an Armstrong Number	$(1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36)$, not equal to 123.
9474	Armstrong Number	$(9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474)$.
-5	Not an Armstrong Number	Negative numbers are not considered Armstrong numbers.
370	Armstrong Number	$(3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370)$, which equals the original number.

EXPLANATION:

An **Armstrong number** is a number that is equal to the sum of its own digits each raised to the power of the number of digits. For example, 153 has three digits, and if you calculate $(1^3 + 5^3 + 3^3)$, the result is 153 itself, so it is an Armstrong number. Similarly, 9474 has four digits, and $(9^4 + 4^4 + 7^4 + 4^4 = 9474)$, making it an Armstrong number too. On the other hand, 123 gives $(1^3 + 2^3 + 3^3 = 36)$, which is not equal to 123, so it is not an Armstrong number.

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

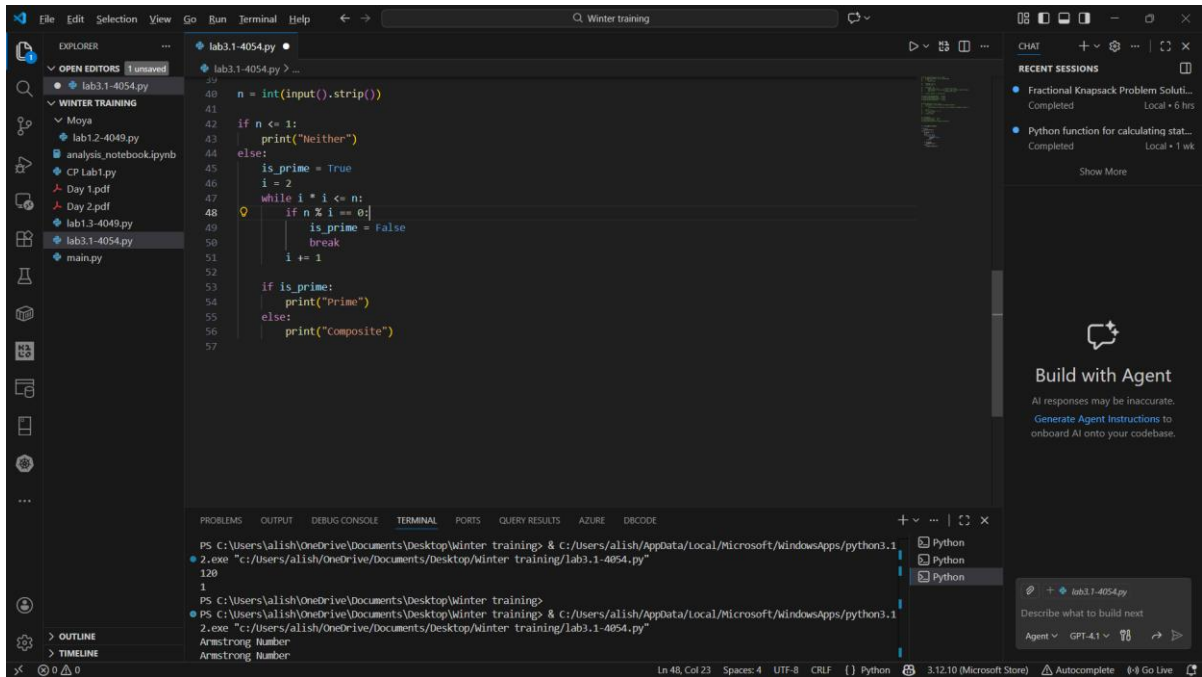
Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.

PROMPT:

Write an optimized Python program that determines whether a given integer is Prime, Composite, or Neither. The program should validate the input, check divisibility only up to $\sqrt{n}$ for efficiency, and print exactly one result. Also, include a brief comparison with the basic divisibility approach.

SCREENSHOT OF GENERATED CODE:



INPUT & OUTPUT:

Input	Output	Reason
11	Prime	No divisors found from 2 to $\sqrt{11}$, so it is a prime number.
15	Composite	Divisible by 3 ($15 \% 3 = 0$), so it is a composite number.
1	Neither	Numbers less than 2 are neither prime nor composite.
-5	Neither	Negative numbers are neither prime nor composite.
2	Prime	Only divisible by 1 and itself; no other divisors are found.

EXPLANATION:

The function first checks if the input is a valid integer. If the value is less than 2, it returns “Neither.” For numbers 2 or greater, it checks divisibility only up to $\sqrt{n}$ for efficiency. If a divisor is found, it returns “Composite”; otherwise, it returns “Prime.”

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

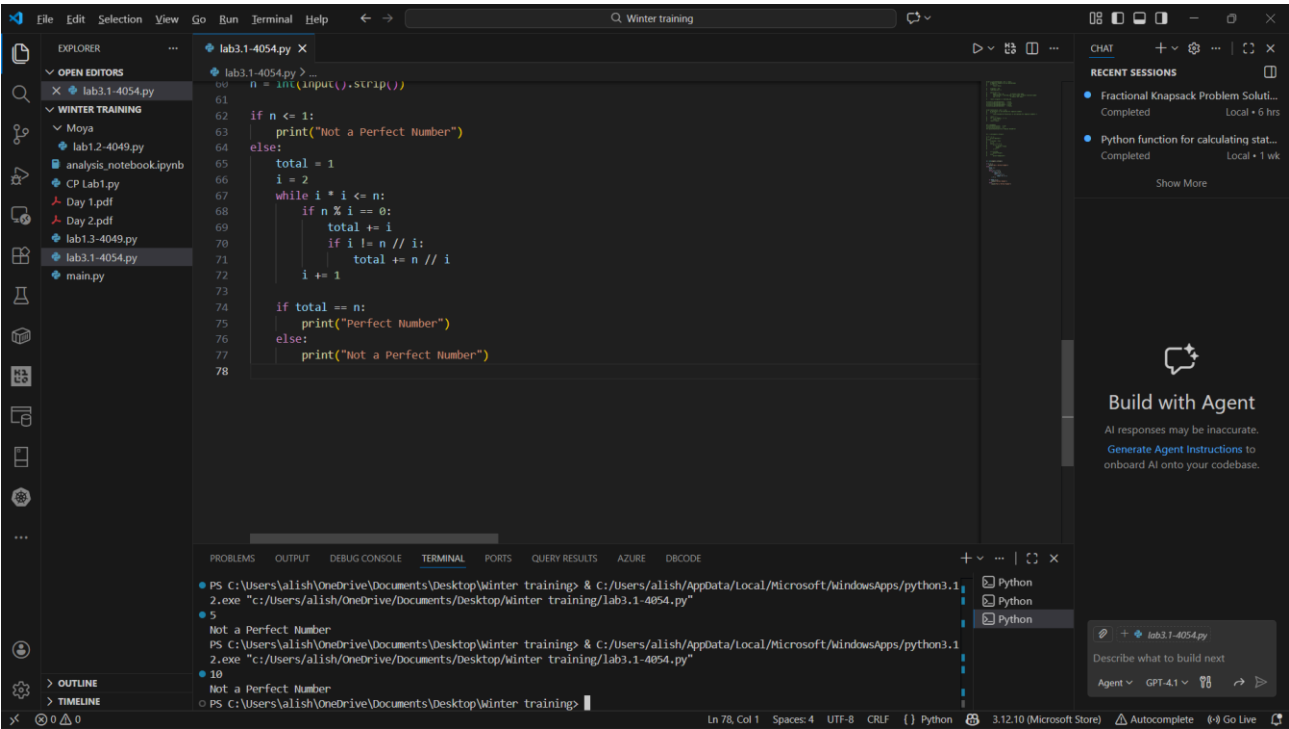
- Record the AI-generated code.

- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

PROMPT:

Write a Python function to determine whether a given integer is a Perfect Number. The function should correctly handle 0, 1, and negative values, use an efficient method to check divisors, and print either Perfect Number or Not a Perfect Number.

SCREENSHOT OF GENERATED CODE:



INPUT & OUTPUT:

Input	Output	Reason
6	Perfect Number	Divisors are 1, 2, 3; sum = 6.
28	Perfect Number	Divisors are 1, 2, 4, 7, 14; sum = 28.
12	Not a Perfect Number	Divisors are 1, 2, 3, 4, 6; sum = 16.
1	Not a Perfect Number	1 has no proper divisors.
-5	Not a Perfect Number	Negative numbers cannot be perfect numbers.

EXPLANATION:

The program first rejects numbers less than or equal to 1. For valid numbers, it finds proper divisors efficiently by checking only up to $\sqrt{n}$ and summing divisor pairs. If the sum of proper divisors equals the number, it prints **Perfect Number**; otherwise, **Not a Perfect Number**.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

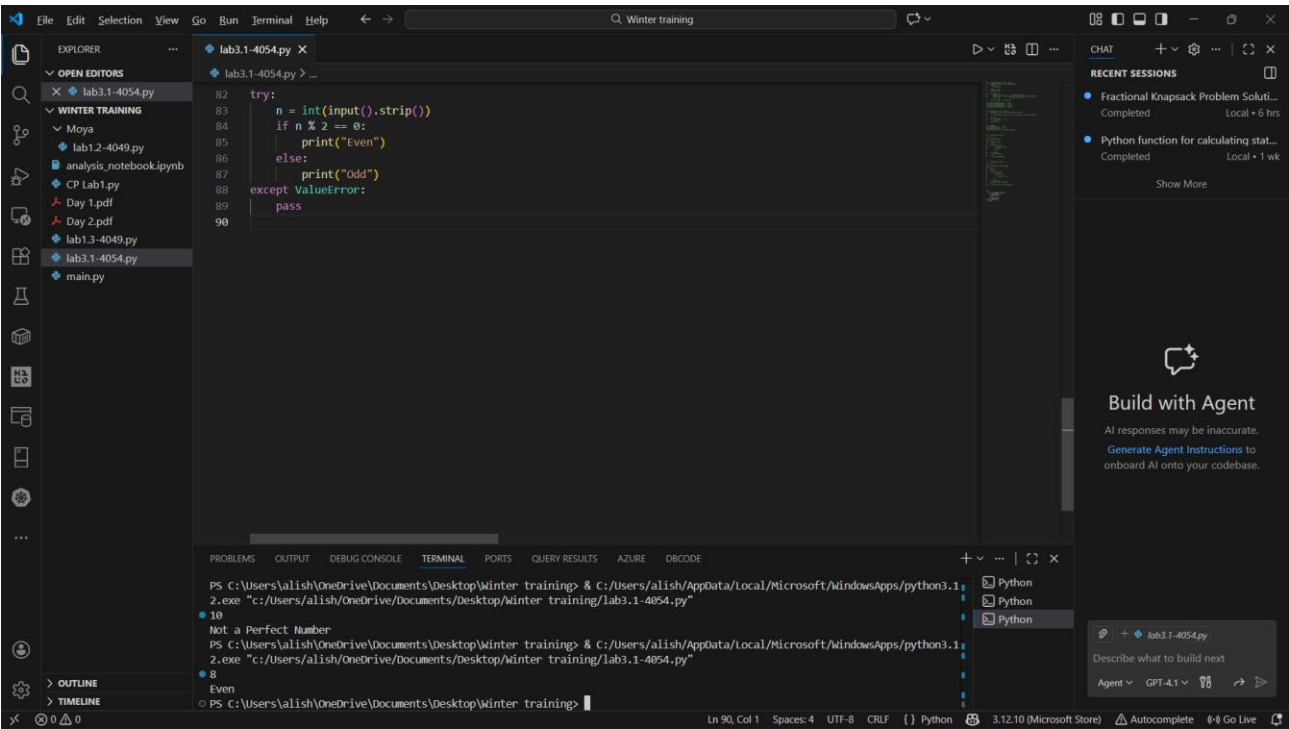
Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

PROMPT:

Write a Python program that determines whether a given number is Even or Odd. The program must validate the input, correctly handle negative and non-integer values, and print only Even or Odd.

SCREENSHOT OF GENERATED CODE:



INPUT & OUTPUT:

Input	Output	Reason
8	Even	8 is divisible by 2 with no remainder.
15	Odd	15 leaves a remainder when divided by 2.

0	Even	0 is divisible by 2.
-4	Even	Negative even numbers are divisible by 2.
-7	Odd	Negative odd numbers leave a remainder when divided by 2.

EXPLANATION:

The program first validates the input to ensure it is an integer.
It then checks divisibility by 2 using the modulo operator.
If the remainder is zero, it prints **Even**; otherwise, it prints **Odd**.